

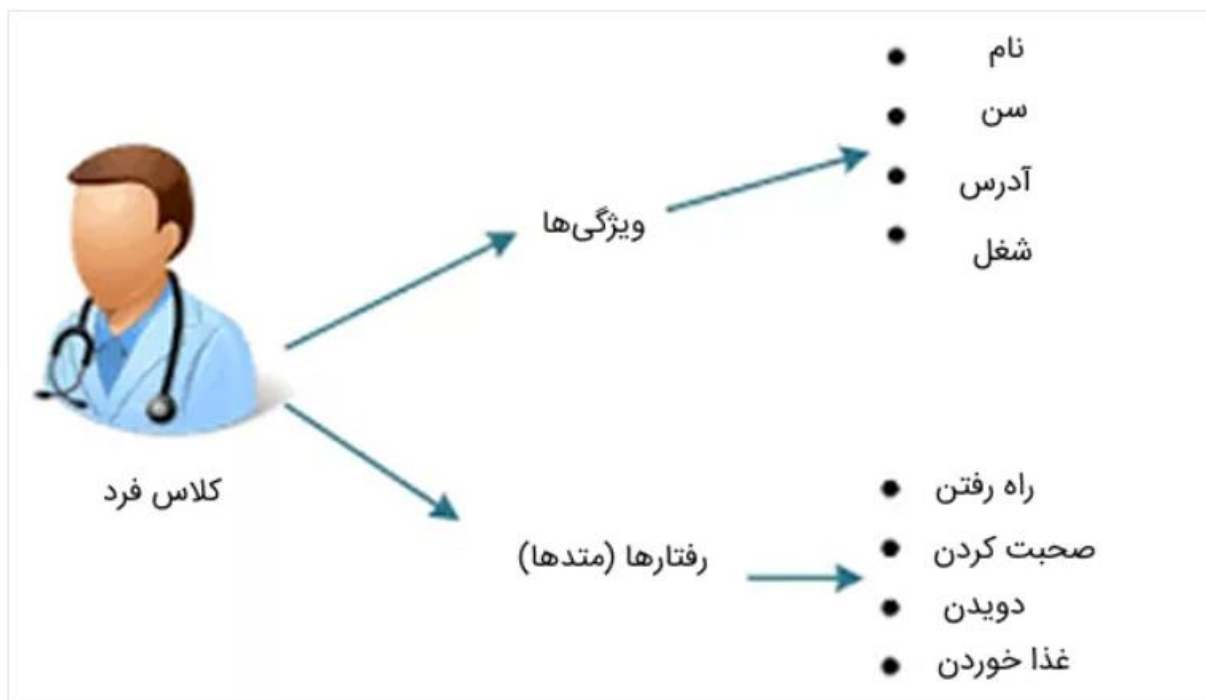
جلسه 15

شی گرای در پایتون چیست ؟

رویکرد برنامه نویسی شی گرا، بر پایه دو مفهوم اصلی **شی (Object)** و **کلاس (Class)** شکل گرفته است که با این دو مفهوم می توان اطلاعات تمام ماهیت های (Entity) دنیای واقعی را در برنامه به راحتی ذخیره کرد.

به بیان جزئی تر می توان گفت هدف از شی گرای در پایتون ساخت کلاسی است که دربرگیرنده تمامی «ویژگی ها» (Properties) و «رفتارها | متدهای (Methods)» مرتبط با یک ماهیت باشد و از طریق ایجاد شی از کلاس مربوطه بتوان به همه «مشخصه ها (Attributes)» یا همان ویژگی ها و متدها دسترسی داشت.

به عنوان مثال، می توان ماهیت یک شخص را در دنیای واقعی در نظر گرفت که این ماهیت شامل مجموعه ای از ویژگی ها و رفتارهای (متدهای) مختلفی نظیر نام، سن، آدرس محل سکونت، راه رفتن، صحبت کردن و دویدن می شود. در برنامه نویسی شی گرا با استفاده از ساخت کلاس می توان این مشخصه ها را تعریف کرد و قالبی را به وجود آورد و سپس برای تعریف افراد مختلف با اطلاعات متفاوت، اشیای مختلفی را از این کلاس ایجاد کرد.



شی و کلاس در پایتون

[انواع داده‌های پایتون](#) نظیر [اعداد](#)، [رشته‌ها](#) و [لیست‌ها](#) به منظور ذخیره مقادیر ساده‌ای از اطلاعات ایجاد شده‌اند. به عنوان مثال، با تعریف متغیرهایی با نام‌های Price و ColorName از نوع عددی و رشته‌ای، می‌توان به ترتیب، قیمت یک سیب و رنگ آن را ذخیره کرد.

یکی از روش‌های ذخیره‌سازی چنین اطلاعاتی در زبان پایتون این است که از ساختار داده لیست استفاده شود و برای هر کارمند در سازمان، لیستی از اطلاعات شخصی بسازید. مثال زیر، اطلاعات سه کارمند را نشان می‌دهد که با استفاده از ساختار داده لیست در سه متغیر جداگانه ذخیره شده‌اند:

```
1 kirk = ["James Kirk", 34, "Captain", 2265]
2 spock = ["Spock", 35, "Science Officer", 2254]
3 mccoy = ["Leonard McCoy", "Chief Medical Officer", 2266]
```

با این که چنین روشی برای ذخیره اطلاعات کارمندان، ساده است، اما مشکلات مهمی دارد. چنانچه بخواهید به اطلاعات هر شخص در برنامه دسترسی داشته باشید، باید از اندیس استفاده شود. به عنوان مثال، اندیس صفر از لیست kirk، نام کارمند را مشخص می‌کند.

زمانی که قطعه کدهای برنامه شما زیاد شوند و چندین برنامه نویس بخواهند با چنین روشی به داده‌ها دسترسی پیدا کنند، روش دسترسی به اطلاعات با استفاده از اندیس، مناسب نیست، زیرا افراد برنامه نویس باید به‌طور مداوم لیست اطلاعات کارمندان را چک کنند و از این مسئله مطمئن شوند که چه اندیسی، در برگیرنده چه اطلاعاتی از کارمندان است.

با این که چنین روشی برای ذخیره اطلاعات کارمندان، ساده است، اما مشکلات مهمی دارد. چنانچه بخواهید به اطلاعات هر شخص در برنامه دسترسی داشته باشید، باید از اندیس استفاده شود. به عنوان مثال، اندیس صفر از لیست kirk، نام کارمند را مشخص می‌کند.

زمانی که قطعه کدهای برنامه شما زیاد شوند و چندین برنامه نویس بخواهند با چنین روشی به داده‌ها دسترسی پیدا کنند، روش دسترسی به اطلاعات با استفاده از اندیس، مناسب نیست، زیرا افراد برنامه نویس باید به‌طور مداوم لیست اطلاعات کارمندان را چک کنند و از این مسئله مطمئن شوند که چه اندیسی، در برگیرنده چه اطلاعاتی از کارمندان است.

به عبارتی می‌توان گفت شی گرای در پایتون این امکان را فراهم می‌کند تا با استفاده از کلاس، بتوان ساختار داده جدیدی را تعریف کرد و برای آن ویژگی‌ها و متدهای مختلفی در نظر گرفت. کلاس، طرح اولیه‌ای از هر ماهیت است و داده‌ای را در خود ذخیره نمی‌کند. به منظور ذخیره کردن داده‌ها با استفاده از کلاس، باید نمونه‌ای (وهله Instance) از کلاس ساخته شود تا بتوان داده‌های خود را در آن ذخیره کرد. به هر نمونه ایجاد شده از کلاس، شی گفته می‌شود. در واقع کلاس «نقشه ساخت» شی به حساب می‌آید.

نحوه تعریف کلاس و شی در پایتون

به منظور تعریف کلاس در پایتون، از کلمه کلیدی `class` استفاده می‌شود. پس از نوشتن این کلمه کلیدی، باید نام کلاس درج شود. برای نوشتن حرف اول نام کلاس باید از حروف بزرگ انگلیسی استفاده کنیم. در نهایت، علامت دو نقطه (:) پس از نام کلاس قرار می‌گیرد. در ادامه، مثالی از نحوه ساخت کلاس برای کلاس `Person` ملاحظه می‌شود.

```
1 class Person:
2     pass
```

چنانچه نام کلاس از چندین کلمه تشکیل شده باشد، باید کلمات بدون درج فاصله به گونه‌ای نوشته شوند که حروف نخست کلمات، حروف بزرگ انگلیسی باشند. به عنوان مثال، اگر قصد دارید کلاسی برای کارمندان بخش فروش سازمان بسازید، می‌توانید از نام `SalesEmployee` استفاده کنید.

از کلمه کلیدی `pass` در توابع و کلاس‌هایی استفاده می‌شود که دستورات در بدنه آن‌ها نوشته نشده باشند. نوشتن این کلمه کلیدی باعث می‌شود در زمان اجرای برنامه، خطای کامپایلر رخ ندهد. پس از ساخت کلاس، می‌توان شیئی را به عنوان وهله‌ای از کلاس ساخت. در بخش پیشین، از متغیری با عنوان `kirk` برای ذخیره اطلاعات یکی از کارمندان استفاده کردیم. از همین نام برای ساخت یک شی جدید از کلاس `Person` به روش زیر استفاده می‌کنیم:

```
1 class Person:
2     pass
3
4 kirk = Person()
```

چنانچه کلاس `Person` دارای ویژگی‌ها و متدهای مختلف باشد، با استفاده از شی `kirk` می‌توان به هر یک از آن‌ها دسترسی داشت و اطلاعات کارمند `kirk` را با استفاده از ویژگی‌ها و متد کلاس `Person` ذخیره کرد. با استفاده از دستور `type()` نیز می‌توان کلاس شی تعریف شده را مشخص کرد. در مثال زیر، نحوه کاربرد این تابع ارائه شده است:

```
1 print(type(kirk))
```

خروجی قطعه کد بالا به صورت `<class '__main__.Person'>` است که این خروجی نشان می‌دهد شی `kirk` به عنوان نمونه‌ای از کلاس `Person` محسوب می‌شود. پس از ساخت کلاس `Person` و تعریف شی از این کلاس، می‌توان با استفاده از تابع `isinstance()` مشخص کرد که آیا شی، نمونه‌ای از کلاس `Person` است؟ قطعه کد زیر، نحوه استفاده از این تابع را نشان می‌دهد.

```
1 kirk = Person()
2 print(isinstance(kirk, Person)) # True
```

خروجی قطعه کد بالا برابر با `True` است که این مقدار نشان می‌دهد شی `kirk`، نمونه‌ای از کلاس `Person` محسوب می‌شود.

تعریف ویژگی‌های کلاس در پایتون

شی‌گرایی در پایتون این امکان را برای برنامه‌نویسان فراهم می‌کند تا ویژگی‌های مختلفی را درون کلاس ایجاد کنند. برای ساخت ویژگی می‌توان از تعریف [متغیر](#) استفاده کرد.

دو نوع ویژگی می‌توانیم در کلاس پایتون بسازیم که در ادامه به آن‌ها اشاره شده است:

- متغیرهای کلاس (Class Variables)
- متغیرهای وهله‌ای (Instance Variables)

متغیر کلاس در پایتون چیست ؟

متغیرهای کلاس، نوعی از ویژگی‌های درون کلاس هستند که پس از اعلان کلاس تعریف می‌شوند. این متغیرها خارج از توابع (متدهای) داخل کلاس قرار می‌گیرند. مقادیر این نوع متغیرها برای تمامی اشیای ساخته شده از کلاس، به اشتراک گذاشته می‌شوند.

در قطعه کد زیر، متغیرهای `extension` و `version` مثال‌هایی از متغیرهای کلاس `HtmlDocument` هستند.

```
1 class HtmlDocument:
2     extension = 'html'
3     version = '5'
```

به منظور دسترسی به مقادیر متغیرهای کلاس، می‌توان از نام کلاس `HtmlDocument` به همراه علامت نقطه (`.`) و سپس نام متغیر کلاس استفاده کرد.

```
1 class HtmlDocument:
2     extension = 'html'
3     version = '5'
4
5 print(HtmlDocument.extension) # html
6 print(HtmlDocument.version) # 5
```

همچنین، می‌توان شیئی از کلاس `HtmlDocument` ساخت و با استفاده از نام شی به متغیرهای کلاس دسترسی داشت.

```
1 class HtmlDocument:
2     extension = 'html'
3     version = '5'
4
5 object = HtmlDocument()
6 print(object.extension) # html
7 print(object.version) # 5
```

چنانچه، نام متغیر کلاسی را به اشتباه فراخوانی کنیم و آن متغیر در کلاس تعریف نشده باشد، خطای کامپایلر رخ می‌دهد.

```
1 class HtmlDocument:
2     extension = 'html'
3     version = '5'
4
5 print(HtmlDocument.media_type)
```

در قطعه کد بالا، متغیر `media_type` در کلاس `HtmlDocument` تعریف نشده است. به همین خاطر، در زمان اجرای قطعه کد بالا، خطای زیر رخ می‌دهد:

```
AttributeError: type object 'HtmlDocument' has no attribute 'media_type'
```

متغیرهای وهله ای در کلاس های پایتون

متغیرهای نمونه، متغیرهایی هستند که درون توابع (متدها) و «سازنده‌ها (Constructors)» تعریف می‌شوند. مقادیر این متغیرها برای هر یک از اشیای ساخته شده از یک کلاس مشابه، متفاوت هستند.

به عبارتی، این نوع متغیرها مختص اشیاء هستند و مقادیرشان بین اشیاء مختلف کلاس به اشتراک گذاشته نمی‌شوند. به منظور مقداردهی اولیه متغیرهای نمونه، می‌توان از متد `__init__` در پایتون درون کلاس استفاده کرد. زمانی که شی جدیدی را از یک کلاس ایجاد می‌کنید، به‌طور خودکار، متد `__init__` فراخوانی می‌شود. بدین ترتیب، می‌توان از این متد برای مقداردهی اولیه مشخصه‌های اشیاء استفاده کرد. در قطعه کد زیر، مثالی از نحوه مقداردهی اولیه متغیرهای نمونه در کلاس `Person` ملاحظه می‌شود.

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6 object_person = Person('John', 25)
7 print(f"I'm {object_person.name}. I'm {object_person.age} years old.")
```

در مثال بالا، متغیرهای `name` و `age` متغیرهای نمونه از کلاس `Person` هستند و به هنگام ساخت شی `object_person` تابع `__init__` فراخوانی می‌شود و مقادیر `John` و `25` درون متغیرهای نمونه `name` و `age` قرار می‌گیرند. همان‌طور که در این مثال ملاحظه می‌کنید، نخستین پارامتر تابع `__init__`، مقدار `self` است. در زبان برنامه نویسی پایتون، زمانی که با استفاده از نام شی، تابعی (متدی) را فراخوانی می‌کنیم، به‌طور خودکار، خود شی به عنوان اولین پارامتر، به آن تابع ارسال می‌شود.

بنابراین، دستور `Person('John',25)` در قطعه کد بالا، معادل دستور `Person(object_person, 'John',25)` است. به همین دلیل، اولین پارامتر تابع درون کلاس، باید پارامتر `self` باشد که به شی فعلی کلاس اشاره دارد. می‌توان نام `self` را تغییر داد، اما به منظور فهم دقیق‌تر کدها و پایبند بودن به قراردادهای زبان پایتون، بهتر است که از همین کلمه کلیدی برای اشاره به شی استفاده شود. خروجی قطعه کد بالا در ادامه ملاحظه می‌شود:

```
I'm John. I'm 22 years old.
```

تعریف متدهای کلاس در پایتون

درون کلاس علاوه بر ویژگی، می توان توابعی را نیز تعریف کرد که عملیات مختلفی را انجام می دهند. به توابع درون کلاس، متد گفته می شود.

انواع متدهای درون کلاس به شرح زیر هستند:

- متد وهله ای (Instance Method)

- متد کلاس (Class Method)

متد وهله ای در کلاس پایتون چیست ؟

شی گرای در پایتون این امکان را برای برنامه نویسان فراهم می کند تا متدهایی را درون کلاس تعریف کنند تا بنا به نیاز، از طریق شی ساخته شده در خارج از کلاس، فراخوانده شوند. به این نوع متدها، متدهای نمونه یا همان متدهای وهله ای گفته می شود.

اولین [آرگومان](#) دریافتی این متدها، عبارت self است که به شی اشاره دارد. در ادامه، قطعه کدی ملاحظه می شود که نحوه تعریف متد نمونه کلاس را نشان می دهد.

```
1 class Request:
2     def send(self):
3         print('Sent')
4
5 http_request = Request()
6 print(http_request.send()) # Sent
```

متد کلاس در پایتون

با استفاده از شی گرای در پایتون می توان متدهایی از نوع متد کلاس تعریف کرد که از طریق آن ها می توان مقادیر متغیرهای کلاس را تغییر داد. همان طور که در بخش مربوط به متغیر کلاس توضیح داده شد، مقادیر این نوع متغیرها برای تمامی اشیاء ساخته شده از یک کلاس مشابه، به اشتراک گذاشته می شوند.

متدهای کلاس را می توان به دو روش در پایتون ایجاد کرد. در اولین روش، از «دکوراتوری (Decorator)» با نام `@classmethod` استفاده می شود که در ادامه قطعه کدی از کاربرد این دکوراتور در ساخت متد کلاس ارائه شده است.

```
1 from datetime import date
2
3 class Student:
4     def __init__(self, name, age):
5         self.name = name
6         self.age = age
7
8     @classmethod
9     def calculate_age(cls, name, birth_year):
10         # calculate age and set it as a age
11         # return new object
12         return cls(name, date.today().year - birth_year)
13
14     def show(self):
15         print(self.name + "'s age is: " + str(self.age))
16
17
18 # create new object using the factory method
19 joy = Student.calculate_age("Joy", 1995)
20 joy.show()
```

در مثال بالا، متد `calculate_age` از نوع متد کلاس است. خاطر نشان می‌شود نخستین پارامتر متد کلاس باید مقدار `cls` باشد که به نام کلاس جاری اشاره می‌کند.

روش دیگری که با کمک آن می‌توان متد کلاس تعریف کرد، استفاده از تابع `classmethod` است. به هنگام استفاده از این تابع، باید متدی را درون کلاس تعریف کرد و نام متد را به عنوان آرگومان تابع `classmethod` در نظر گرفت تا این تابع، متد مشخص شده را به متد کلاس تبدیل کند. در قطعه کد زیر، مثالی از کاربرد تابع `classmethod` برای ساخت متد کلاس `school_name` ملاحظه می‌شود.

```
1 class School:
2     # class variable
3     name = 'ABC School'
4
5     def school_name(cls):
6         print('School Name is :', cls.name)
7
8 # create class method
9 School.school_name = classmethod(School.school_name)
10
11 # call class method
12 School.school_name()
```

مباحث پیشرفته OOP

اصول شی گرای در پایتون

برنامه نویسی شی گرا علاوه بر مفاهیم شی و کلاس، شامل چهار اصل مهم هم هست که با لحاظ کردن این اصول در توسعه پروژه‌های نرم‌افزاری، میزان بهره‌وری برنامه‌ها بیشتر می‌شود و برنامه‌نویسان می‌توانند به راحتی، قطعه کدهای نوشته شده را به روزرسانی و آن‌ها را گسترده‌تر کنند.

- اصل کپسوله‌سازی (Encapsulation)
- اصل انتزاع | تجرید (Abstraction)
- وراثت | ارث‌بری (Inheritance)
- پلی مورفیسم | چندریختی (Polymorphism)

اصل وراثت شی گرایی در پایتون چیست ؟

یکی از اصول شی گرایی در پایتون، اصل وراثت یا ارث بری است که در طی این فرآیند، می توان کلاسی را ایجاد کرد که ویژگی ها و متدهای خود را از کلاس دیگری به ارث ببرد که از قبل تعریف شده است. در این حال، کلاس جدید، کلاس فرزند (Child Class) نام دارد و به کلاسی که مشخصه های خود را به کلاس فرزند می دهد، کلاس والد (Parent Class) گفته می شود.

کلاس فرزند می تواند ویژگی ها و متدهایی را که از کلاس والد به ارث برده است، بنا به نیاز خود تغییر دهد. چنانچه بخواهیم مثال قابل درکی از دنیای واقعی برای اصل وراثت ارائه دهیم، می توان به زن های پدر و مادر و انتقال آن ها به فرزندان اشاره کرد.

ممکن است فرزند رنگ موی خود را از مادرش به ارث ببرد. با این حال، فرزند تصمیم می گیرد رنگ موی خود را تغییر دهد و موی خود را به رنگ صورتی تغییر دهد. با اینکه فرزند، داشتن ویژگی رنگ مو را از پدر و مادر خود به ارث برده است، اما می تواند به تصمیم خود، رنگ آن را تغییر دهد و موی خود را به رنگی درآورد که مشابه رنگ موی پدر و مادر خود نیست.

شی گرایی در پایتون نیز بر اساس تعاریفی که از وراثت و نحوه ارث بری در دنیای حقیقی وجود دارد، این امکان را به برنامه نویس می دهد تا از کلاسی که ساخته شده است، بتوان به دفعات مختلف به شیوه ای استفاده کرد که مشخصه های کلاس، مطابق نیاز کاربر تغییر کنند. به منظور درک بهتر این اصل، مثالی از پایتون ارائه می کنیم.

فرض کنید کلاسی با نام Person ساخته شده است که مشخصه های __init__ و greet را دارد. در قطعه کد زیر، مثالی از این کلاس ملاحظه می شود.

```
1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5     def greet(self):
6         return f"Hi, it's {self.name}"
```

حال می‌خواهیم کلاسی با نام Employee ایجاد کنیم که دقیقاً مشخصه‌های کلاس Person را داشته باشد. در قطعه کد زیر، نحوه ساخت کلاس Person ملاحظه می‌شود.

```
1 class Employee:
2     def __init__(self, name, job_title):
3         self.name = name
4         self.job_title = job_title
5
6     def greet(self):
7         return f"Hi, it's {self.name}"
```

همان‌طور که در قطعه آمده است، کلاس Employee دارای متد greet است که همین متد در کلاس Person نیز وجود دارد. با استفاده از اصل وراثت در شی گرای پایتون، می‌توان متد greet را از کلاس Employee حذف کرد و با ایجاد ارتباط بین دو کلاس Person و Employee، متد greet را از کلاس Person به ارث برد. در این حالت، کلاس Employee به عنوان کلاس فرزند و کلاس Person به عنوان کلاس والد مشخص می‌شوند. در ادامه، نحوه به کارگیری اصل وراثت در پایتون ملاحظه می‌شود.

```
1 class Employee(Person):
2     def __init__(self, name, job_title):
3         self.name = name
4         self.job_title = job_title
```

همان‌طور که در قطعه کد بالا دیده می‌شود، می‌توان در خط اعلان کلاس فرزند، نام کلاس والد را در داخل پرانتز قرار داد. بدین ترتیب، کلاس Employee علاوه بر ویژگی‌های name و job_title، دارای متد greet نیز هست که آن را از کلاس والد خود، یعنی کلاس Person، به ارث می‌برد. در ادامه، مثالی از نحوه دسترسی به متد greet از طریق شی ساخته شده از کلاس Employee ملاحظه می‌شود.

```
1 employee = Employee('John', 'Python Developer')
2 print(employee.greet())
```

خروجی قطعه کد بالا در ادامه نشان داده شده است:

```
Hi, it's John
```

با استفاده از تابع `issubclass()` می‌توان مشخص کرد که آیا دو کلاس رابطه والد - فرزندی دارند؟ در ادامه، نحوه استفاده از این تابع در زبان پایتون در قالب مثال، نشان داده شده است:

```
1 print(issubclass(Employee, Person)) # True
```

همچنین، با استفاده از `super()` می‌توان از داخل کلاس فرزند، به متدهای کلاس والد دسترسی داشت. در ادامه، قطعه کدی از زبان پایتون ارائه شده است که نحوه استفاده از `super()` را نشان می‌دهد.

```
1 class Employee:
2     def __init__(self, name, base_pay, bonus):
3         self.name = name
4         self.base_pay = base_pay
5         self.bonus = bonus
6
7     def get_pay(self):
8         return self.base_pay + self.bonus
9
10
11 class SalesEmployee(Employee):
12     def __init__(self, name, base_pay, bonus, sales_incentive):
13         super().__init__(name, base_pay, bonus)
14         self.sales_incentive = sales_incentive
15
16     def get_pay(self):
17         return super().get_pay() + self.sales_incentive
18
19
20 if __name__ == '__main__':
21     sales_employee = SalesEmployee('John', 5000, 1000, 2000)
22     print(sales_employee.get_pay()) # 8000
```

به عبارتی، تمامی ویژگی‌ها و متدهای یک ماهیت، در یک کلاس تعریف می‌شوند و می‌توان مشخص کرد که با تعریف شی به کدام یک از این مشخصه‌ها می‌توان دسترسی داشت. بر اساس اصل کپسوله‌سازی شی گرایی، می‌توان سه نوع سطح دسترسی به مشخصه‌های درون کلاس اختصاص داد که در ادامه به آن‌ها اشاره شده است:

- مشخصه‌هایی از نوع عمومی (Public)
- مشخصه‌هایی از نوع خصوصی (Private)
- مشخصه‌هایی از نوع محافظت شده (Protected)

مشخصه‌های عمومی کلاس در پایتون

مشخصه‌های عمومی کلاس در شی گرایی به مشخصه‌هایی گفته می‌شود که بدون هیچ محدودیتی بتوان به آن‌ها دسترسی داشت. به عبارتی، ویژگی‌ها و متدهای عمومی کلاس را می‌توان از خارج کلاس (با استفاده از اشیای تعریف شده) یا از طریق کلاس‌های فرزند مدیریت کرد و به مقادیر آن‌ها دسترسی داشت. در قطعه کد زیر، مثالی از مشخصه‌های عمومی کلاس در زبان پایتون ارائه شده است. در این مثال، ویژگی‌های `age` و `name` و متد `print_name` به عنوان مشخصه‌های عمومی کلاس `Person` محسوب می‌شوند که با استفاده از شی `object_person` می‌توان از خارج کلاس، به آن‌ها دسترسی داشت.

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5     def print_name(self):
6         print('The name is :', self.name)
7
8 object_person = Person('John', 25)
9 print(object_person.name)
10 object_person.print_name()
```

مشخصه های محافظت شده کلاس در پایتون

اصل کپسوله سازی شی گرایی این امکان را فراهم می کند تا بتوان مشخصه هایی در کلاس ایجاد کرد که از خارج کلاس، امکان دسترسی به آن ها وجود ندارد. تنها راه دسترسی به مشخصه های محافظت شده، از درون کلاس یا از داخل کلاس های فرزند است. این اصل باعث بالا بردن سطح امنیت در توسعه برنامه می شود به منظور تعریف مشخصه های حفاظت شده در پایتون از علامت (_) استفاده می شود.

در قطعه کد زیر، مثالی از نحوه کاربرد ویژگی های محافظت شده در پایتون ارائه شده است. در این مثال، متغیر `_q` از نوع متغیر محافظت شده است که نام آن با علامت (_) آغاز می شود.

```
1 class Base1:
2     def __init__(self):
3         self._q = "Javatpoint"
4
5 obj_1 = Base1()
6 print(obj_1._q)
```

خروجی قطعه کد بالا، در ادامه آمده است.

```
Javatpoint
```

```
1 class Person:
2     def __init__(self, name):
3         self.name = name
4     def _warmup(self):
5         print("{} is warming up".format(self.name))
6 jack = Person ('Jack')
7 jack._warmup()
```

خروجی قطعه کد بالا در ادامه ملاحظه می شود.

```
Jack is warming up
```

مشخصه های خصوصی کلاس در پایتون

مشخصه های خصوصی در شی گرایی نیز دارای محدودیت های دسترسی هستند. به عبارتی، به این نوع مشخصه ها نمی توان از خارج کلاس دسترسی داشت. علاوه بر این، کلاس های فرزند نیز محدودیت دسترسی به مشخصه های خصوصی کلاس والد خود را دارند. بدین ترتیب، مشخصه های خصوصی تنها از درون کلاس قابل دسترس هستند. به منظور تعریف ویژگی ها و متدهای خصوصی در پایتون از علامت (__) در ابتدای نام مشخصه ها استفاده می شود.

```
1 class Base1:
2     def __init__(self):
3         self.__q = "Javatpoint"
4
5 obj_1 = Base1()
6 print(obj_1.__q)
```

خروجی قطعه کد بالا، در ادامه ملاحظه می شود.

Javatpoint